

弹簧系统仿真实验

宁尚哲 PB24000099

May 17, 2026

Abstract

1 基础部分

1.1 原理与代码实现

1.1.1 半隐式时间积分

数学原理:

半隐式时间积分 (Semi-implicit Euler) 是一种显式的时间积分方法, 其核心思想是在计算速度时使用当前时刻的位置, 但在更新位置时使用新计算的速度。对于弹簧质点系统, 我们有以下公式:

$$\mathbf{x}^{n+1} = \mathbf{x}^n + h\mathbf{v}^{n+1} \quad (1)$$

$$\mathbf{v}^{n+1} = \mathbf{v}^n + h\mathbf{M}^{-1}(\mathbf{f}_{\text{int}}(\mathbf{x}^n) + \mathbf{f}_{\text{ext}}) \quad (2)$$

其中 $\mathbf{x}^{n+1} \in \mathbb{R}^{3n \times 1}$ 是所有顶点位置组成的向量, $\mathbf{v}^{n+1} \in \mathbb{R}^{3n \times 1}$ 是所有顶点速度组成的向量, h 是时间步长, $\mathbf{M} \in \mathbb{R}^{3n \times 3n}$ 是质量矩阵, $\mathbf{f}_{\text{int}}$ 是内部弹性力, $\mathbf{f}_{\text{ext}}$ 是外力 (如重力)。

根据能量观点, 弹簧系统的弹性能量为:

$$E = \sum_i \frac{k}{2} (\|\mathbf{x}_i\| - L_i)^2 \quad (3)$$

其中 $\mathbf{x}_i = \mathbf{x}_{i1} - \mathbf{x}_{i2}$ 是第 i 根弹簧两端顶点的位置差, L_i 是弹簧的原长, k 是劲度系数。

弹性力是能量的负梯度:

$$\mathbf{f}_{\text{int}} = -\nabla E = -\sum_i k(\|\mathbf{x}_i\| - L_i) \frac{\mathbf{x}_i}{\|\mathbf{x}_i\|} \quad (4)$$

代码实现:

在代码中, 我们首先计算弹性力的梯度 (即负的弹性力):

```
// Compute gradient of elastic energy
for each edge e:
    edge_vector = X[e.first] - X[e.second]
    force = k * (|edge_vector| - L) *
        edge_vector / |edge_vector|
    g[e.first] += force; g[e.second] -=
        force
```

然后在时间积分步骤中实现半隐式欧拉方法:

```
// Semi-implicit Euler integration
acceleration = -computeGrad(stiffness) +
    acceleration_ext
vel += h * acceleration
X += h * vel
if (enable_damping) vel *= damping
```

1.1.2 隐式时间积分

数学原理:

隐式时间积分 (Implicit Euler) 在计算速度时使用下一时刻的位置, 这使得系统更加稳定, 可以使用更大的时间步长。其公式为:

$$\mathbf{x}^{n+1} = \mathbf{x}^n + h\mathbf{v}^{n+1} \quad (5)$$

$$\mathbf{v}^{n+1} = \mathbf{v}^n + h\mathbf{M}^{-1}(\mathbf{f}_{\text{int}}(\mathbf{x}^{n+1}) + \mathbf{f}_{\text{ext}}) \quad (6)$$

由于 $\mathbf{f}_{\text{int}}(\mathbf{x}^{n+1})$ 依赖于未知的 $\mathbf{x}^{n+1}$, 我们需要将其转化为一个关于 $\mathbf{x}^{n+1}$ 的优化问题。整理后得到:

$$\mathbf{x}^{n+1} = \mathbf{x}^n + h\mathbf{v}^n + h^2\mathbf{M}^{-1}(-\nabla E(\mathbf{x}^{n+1}) + \mathbf{f}_{\text{ext}}) \quad (7)$$

定义 $\mathbf{y} := \mathbf{x}^n + h\mathbf{v}^n + h^2\mathbf{M}^{-1}\mathbf{f}_{\text{ext}}$, 则上述公式可以转化为优化问题:

$$\min_{\mathbf{x}} g(\mathbf{x}) = \frac{1}{2h^2}(\mathbf{x} - \mathbf{y})^\top \mathbf{M}(\mathbf{x} - \mathbf{y}) + E(\mathbf{x}) \quad (8)$$

这是一个无约束优化问题, 其一阶最优条件 (KKT条件) 为:

$$\nabla g(\mathbf{x}) = \frac{1}{h^2}\mathbf{M}(\mathbf{x} - \mathbf{y}) + \nabla E(\mathbf{x}) = \mathbf{0} \quad (9)$$

使用牛顿法求解这个优化问题:

$$\mathbf{x}^{n+1} = \mathbf{x}^n - (\nabla^2 g)^{-1} \nabla g(\mathbf{x}^n) \quad (10)$$

其中 $\nabla^2 g$ 是能量 g 的 Hessian 矩阵:

$$\nabla^2 g = \frac{1}{h^2}\mathbf{M} + \mathbf{H} \quad (11)$$

$\mathbf{H}$ 是弹性能量 E 的 Hessian 矩阵。对于单根弹簧, 其 Hessian 为:

$$\mathbf{H}_i = k \frac{\mathbf{x}_i \mathbf{x}_i^\top}{\|\mathbf{x}_i\|^2} + k \left(1 - \frac{L}{\|\mathbf{x}_i\|}\right) \left(\mathbf{I} - \frac{\mathbf{x}_i \mathbf{x}_i^\top}{\|\mathbf{x}_i\|^2}\right) \quad (12)$$

总的 Hessian 矩阵 $\mathbf{H} \in \mathbb{R}^{3n \times 3n}$ 是所有弹簧 Hessian 按照顶点索引组装起来的稀疏矩阵。

代码实现:

首先计算弹性能量的 Hessian 矩阵:

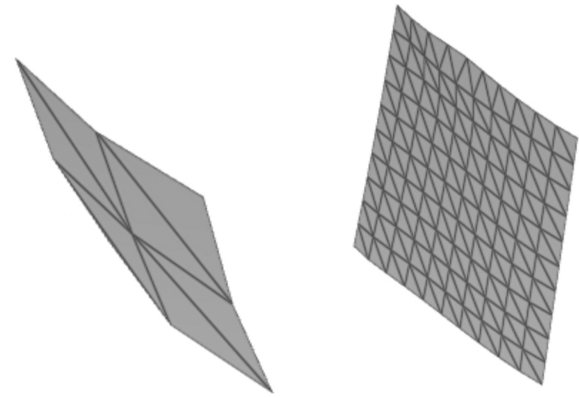
```
// Compute Hessian matrix for elastic energy
for each edge e:
    edge_vector = X[e.first] - X[e.second]
    normalized = edge_vector / |edge_vector|
    Hi = k * (normalized * normalized^T)
    if (|edge_vector| > L):
        Hi += k * (1 - L/|edge_vector|) * (I
            - normalized * normalized^T)
// Assemble Hi to global Hessian matrix
```

然后实现隐式时间积分：

```

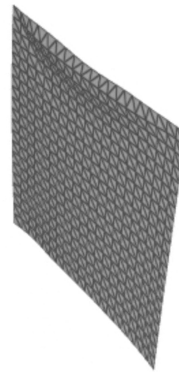
1 // Implicit Euler integration
2 Y = X + h*vel + h^2*acceleration_ext
3 grad_g = (1/h^2)*M*(X - Y) + computeGrad(
4   stiffness)
5 H_g = (1/h^2)*M + computeHessianSparse(
6   stiffness)
7
8 // Apply Dirichlet boundary conditions
9 set grad_g to 0 for fixed vertices
10 set H_g diagonal to 1 for fixed DOFs
11
12 // Solve linear system and update
13 solve H_g * delta_X = -grad_g
14 X += delta_X
15 vel = (X - X_old) / h

```

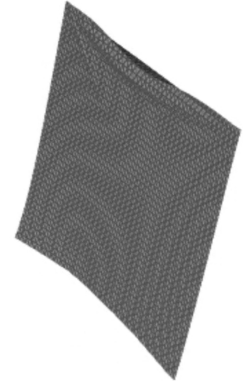


(a) 2x2网格

(b) 4x4网格



(c) 10x10网格



(d) 20x20网格

Figure 1: 半隐式时间积分在不同分辨率网格下的仿真结果

1.2 不同参数对比下的实现效果

1.2.1 半隐式时间积分

实验参数设置如下表所示：

Table 1: 半隐式时间积分实验参数

参数	数值
stiffness	10000
h	0.033
damping	0.995
gravity	9.8

我们测试了不同分辨率的网格，包括2x2, 10x10, 20x20, 40x40。实验结果表明，半隐式时间积分在小时间步长下能够稳定运行，但随着时间步长增大会出现数值不稳定现象。

这是一个stiffness较大的测试，但是质量比较小，感觉是没有那么重但是也舒展不开，因为刚性比较大。适合模拟那种弹力布套等。

1.2.2 隐式时间积分

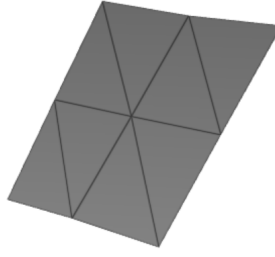
实验参数设置如下表所示：

Table 2: 隐式时间积分实验参数

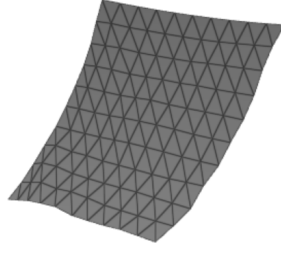
参数	数值
stiffness	10000
h	0.033
damping	0.995
gravity	-98

隐式时间积分使用与半隐式相同的参数设置。实验结果表明，隐式时间积分在相同的时间步长下比半隐式更加稳定，可以使用更大的时间步长而不出现数值不稳定现象。

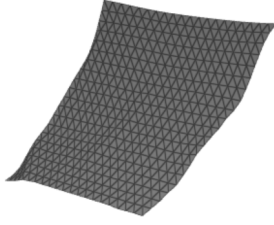
这个参数就是比半隐式还有变重了！但是看起来就非常生硬，因为强度调的依旧很高，所以掉不太下来，但是又很沉，硬度又很大，适合模拟那种干瘪的纸张、脆的。



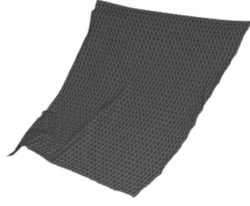
(a) 2x2网格



(b) 4x4网格



(c) 10x10网格



(d) 20x20网格

Figure 2: 隐式时间积分在不同分辨率网格下的仿真结果

性能对比:

隐式时间积分虽然每步计算时间较长（需要求解线性方程组），但由于可以使用更大的时间步长，在达到相同仿真精度时总体计算时间可能更短。对于高分辨率网格（如20x20），隐式方法的优势更加明显。

2 选做部分

2.1 与球之间的碰撞

2.1.1 原理与实现

数学原理:

为了模拟弹簧质点系统与球体的碰撞，我们采用基于惩罚力的方法。当顶点进入球体的碰撞区域时，会产生一个向外的推力，使顶点回到球体外部。

定义球心为 $\mathbf{c}$ ，球半径为 r ，碰撞半径放大系数为 s （通常取1.1），惩罚系数为 k^{penalty} 。对于任意顶点 $\mathbf{x}$ ，碰撞力为：

$$\mathbf{f} = k^{\text{penalty}} \max(sr - \|\mathbf{x} - \mathbf{c}\|, 0) \frac{\mathbf{x} - \mathbf{c}}{\|\mathbf{x} - \mathbf{c}\|} \quad (13)$$

其中 $\frac{\mathbf{x} - \mathbf{c}}{\|\mathbf{x} - \mathbf{c}\|}$ 是从球心指向顶点的单位向量，确保力的方向正确。 $\max(sr - \|\mathbf{x} - \mathbf{c}\|, 0)$ 确保只有当顶点进入碰撞区域时才产生力。

碰撞半径放大系数 s 的作用是在实际未发生接触时就产生接触力，以减少视觉上的穿透现象。

代码实现:

在代码中，我们实现了球体碰撞力的计算函数：

```
// Compute sphere collision force
for each vertex v:
    dist = |X[v] - sphere_center|
    if dist < collision_dist:
        normal = (X[v] - sphere_center) / dist
        penetration = collision_dist - dist
        force[v] = k_penalty * penetration * normal
```

在时间积分步骤中，我们将碰撞力转换为加速度并添加到系统中：

```
// Integrate collision force
acc_collision = getSphereCollisionForce() / mass_per_vertex

// In implicit Euler: Y += h^2 * acc_collision
// In semi-implicit Euler: acceleration += acc_collision
```

2.1.2 效果

球体碰撞实验参数设置如下表所示：

Table 3: 球体碰撞实验参数

参数	数值
stiffness	10000
h	0.033
damping	0.995
gravity	9.8
collision_penalty_k	2000
collision_scale_factor	1.1
sphere_radius	0.4
sphere_center	(0, -0.5, -0.5)

我们使用20x20网格测试了球体碰撞效果。实验结果表明，基于惩罚力的碰撞检测方法能够有效防止布料穿透球体，碰撞响应平滑且物理合理。就好像是一个布料掉下来，右边是两个固定点，很自然。

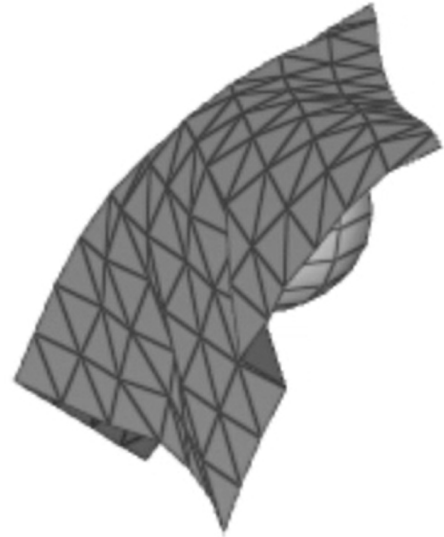


Figure 3: 球体碰撞仿真结果（20x20网格）

碰撞效果分析:

- 碰撞检测准确性:** 当顶点进入碰撞区域时，系统能够及时检测并产生向外的推力。
- 力的方向正确性:** 碰撞力始终沿着从球心指向顶点的方向，确保顶点被推离球体。
- 参数敏感性:** 惩罚系数 k^{penalty} 和碰撞半径放大系数 s 对碰撞效果影响较大。较大的惩罚系数会产生更强的碰撞力，但可能导致数值不稳定；较大的放大系数可以提前产生碰撞力，减少视觉穿透。

2.2 Liu加速方法

2.2.1 原理与代码实现

数学原理:

Liu等人提出的快速弹簧质点系统仿真方法通过引入辅助变量 $\mathbf{d}$ 来加速求解。其核心思想是将弹性能量重新表述,然后使用Local-Global交替求解策略。

能量重新表述:

原始弹性能量为:

$$E = \sum_i \frac{k}{2} (\|\mathbf{x}_i\| - L_i)^2 \quad (14)$$

其中 $\mathbf{x}_i = \mathbf{x}_{i1} - \mathbf{x}_{i2}$ 是第 i 根弹簧两端顶点的位置差。

Liu等人发现, $(\|\mathbf{x}_i\| - L_i)^2$ 可以看作是一个优化问题的解:

$$\min_{\|\mathbf{d}\|=L} \|\mathbf{x}_i - \mathbf{d}\|^2 \quad (15)$$

因此,我们可以将弹性能量重新表述为:

$$E = \frac{1}{2} \sum_i k_i \|\mathbf{x}_i - \mathbf{d}_i\|^2 \quad (16)$$

将二范数展开并整理为矩阵形式:

$$E = \frac{1}{2} \mathbf{x}^\top \mathbf{L} \mathbf{x} - \mathbf{x}^\top \mathbf{J} \mathbf{d} \quad (17)$$

其中 $\mathbf{L} \in \mathbb{R}^{3n \times 3n}$ 是拉普拉斯矩阵, $\mathbf{J} \in \mathbb{R}^{3n \times 3s}$ 是关联矩阵, n 是顶点个数, s 是弹簧个数。

Local-Global求解策略:

结合隐式欧拉积分的能量函数,我们需要优化的总能量为:

$$\min_{\mathbf{x}, \mathbf{d}} g(\mathbf{x}) = \min_{\mathbf{x}, \mathbf{d}} \frac{1}{2h^2} (\mathbf{x} - \mathbf{y})^\top \mathbf{M} (\mathbf{x} - \mathbf{y}) + \frac{1}{2} \mathbf{x}^\top \mathbf{L} \mathbf{x} - \mathbf{x}^\top \mathbf{J} \mathbf{d} \quad (18)$$

整理后得到:

$$\min_{\mathbf{x}, \mathbf{d}} \frac{1}{2} \mathbf{x}^\top (\mathbf{M} + h^2 \mathbf{L}) \mathbf{x} - \mathbf{x}^\top (h^2 \mathbf{J} \mathbf{d} + \mathbf{M} \mathbf{y}) \quad (19)$$

Local Step (局部步骤):

固定 $\mathbf{x}$, 对每根弹簧求解 $\mathbf{d}_i$:

$$\mathbf{d}_i = L_i \frac{\mathbf{x}_i}{\|\mathbf{x}_i\|} \quad (20)$$

这个步骤非常简单,可以并行计算。

Global Step (全局步骤):

固定 $\mathbf{d}$, 求解 $\mathbf{x}$ 。对能量求导并令导数为0, 得到线性方程组:

$$\mathbf{A} \mathbf{x} = \mathbf{b} \quad (21)$$

其中 $\mathbf{A} = \mathbf{M} + h^2 \mathbf{L} \in \mathbb{R}^{3n \times 3n}$, $\mathbf{b} = h^2 \mathbf{J} \mathbf{d} + \mathbf{M} \mathbf{y} \in \mathbb{R}^{3n}$ 。

关键性质: 矩阵 $\mathbf{A}$ 只与系统拓扑有关,在仿真过程中保持不变,因此可以预先分解,每次迭代只需要更新右端项 $\mathbf{b}$ 。

代码实现:

```
// Fast Mass-Spring: Initialization (once)
for each vertex v: A[v*3+d, v*3+d] =
    mass_per_vertex
for each edge (i, j):
    A[i*3+d, i*3+d] += h^2 * k; A[j*3+d, j
        *3+d] += h^2 * k
    A[i*3+d, j*3+d] -= h^2 * k; A[j*3+d, i
        *3+d] -= h^2 * k
apply Dirichlet BC and prefactorize A

// Each time step: Local-Global iteration
Y = X + h*vel + h^2*acceleration_ext
for iter = 1 to max_iter:
    // Local: d[i] = L * (X[i] - X[j]) / |X[
        i] - X[j]|
    // Global: b = M*Y + h^2*J*d, solve A*X
        = b
    // Enforce fixed vertex positions
    vel = (X - X_old) / h.
```

2.2.2 实现效果

Liu加速方法实验参数设置如下表所示:

Table 4: Liu加速方法实验参数

参数	数值
stiffness	300
h	0.033
damping	0.995
gravity	-98
max_iter	1、5、20、100

我们测试了不同迭代次数对仿真结果的影响,包括1次、5次、20次和100次迭代(注意截图都是ruzino里面时间=0.5的时候)。实验结果表明,随着迭代次数增加,仿真结果逐渐接近牛顿法求解的结果。

我发现其他参数一致,只改变迭代次数,在相同时间模拟的位置是不一样的,随着迭代次数增大他下落的似乎变快,而且更加自然。低的迭代步数有“角”很不自然。

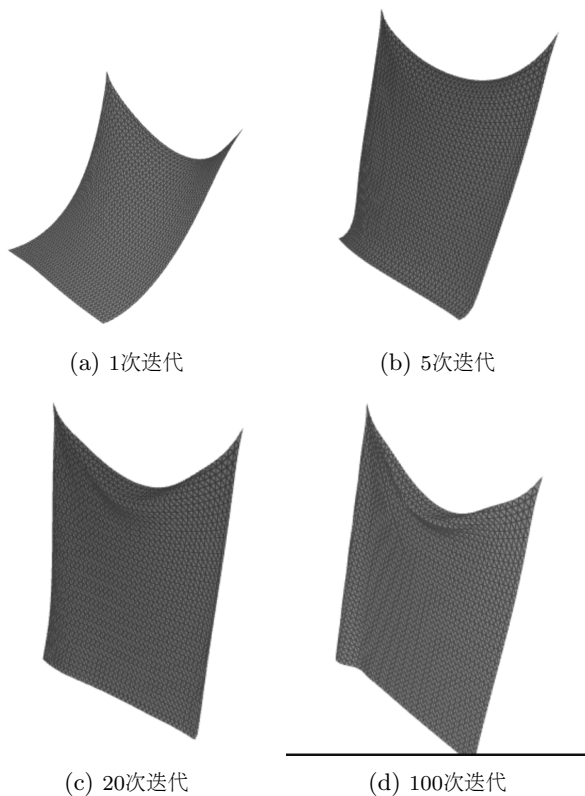


Figure 4: Liu加速方法在不同迭代次数下的仿真结果 (20x20网格)

性能分析:

1. **计算速度:** Liu加速方法通过矩阵预分解, 每次迭代只需要更新右端项并求解线性方程组, 大大提高了计算效率。
2. **收敛性:** 随着迭代次数增加, 仿真结果逐渐收敛到精确解。实验表明, 10次迭代已经能够获得较好的近似结果。
3. **下落速度问题:** 在相同时间参数下, Liu加速方法的布料下落速度比标准隐式方法慢。这可能与参数设置有关, 特别是时间步长 h 和劲度系数 $stiffness$ 的搭配。需要进一步调整参数以获得与标准方法一致的物理行为。
4. **与牛顿法对比:** Liu加速方法在迭代次数较少时计算速度明显快于牛顿法, 但在达到相同精度时可能需要更多迭代。总体而言, Liu方法在计算效率和精度之间提供了良好的平衡。

2.3 体网格

2.3.1 原理与代码实现

数学原理:

体网格 (Volume Mesh) 的弹簧质点仿真与表面网格类似, 但具有以下区别:

1. **拓扑结构不同:** 体网格使用四面体 (Tetrahedral) 或六面体 (Hexahedral) 等体积单元, 而表面网格使用三角形。
2. **弹簧连接方式不同:** 在体网格中, 除了表面边的弹簧连接外, 还有内部边的弹簧连接, 以及可能的对角线连接。
3. **物理行为不同:** 体网格能够模拟体积变形和体积保持, 而表面网格主要模拟表面变形。

对于四面体网格, 我们可以定义以下类型的弹簧连接:

- **边弹簧:** 连接四面体的每条边, 模拟拉伸和压缩。
- **对角线弹簧:** 连接四面体的对角线, 增强体积保持能力。
- **体积弹簧:** 基于四面体体积的约束, 模拟体积保持。

代码实现:

体网格的代码实现与表面网格类似, 主要区别在于边的构建方式:

其余的时间积分、碰撞检测等部分与表面网格完全相同, 可以直接复用。

2.3.2 实现效果

体网格实验参数设置如下表所示:

Table 5: 体网格实验参数

参数	数值
stiffness	50
h	0.033
damping	0.995
gravity	-98

我们使用四面体网格测试了体网格的弹簧质点仿真。实验结果表明, 体网格能够模拟体积变形和体积保持, 与表面网格相比具有不同的物理行为。

我调参发现, 如果 $stiffness$ 太大几乎不会动, 顶点质量不够也不会向下扭曲。所以 $stiffness$ 要小, 而且质量要很大才行。我是固定了一个 $x = 0$ 的面, 其他面是自由可以弯曲的, 很类似与软体机器人的仿真!

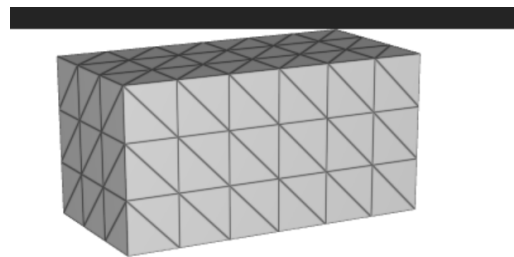


Figure 5: 体网格仿真结果 (3x3x6)

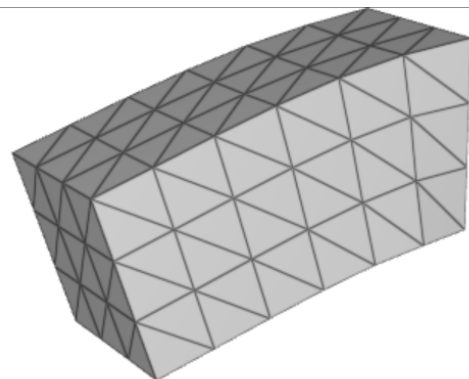


Figure 6: 体网格仿真结果 (1s后)

与表面网格对比:

1. **拓扑结构**: 体网格使用四面体单元, 而表面网格使用三角形面片。
2. **物理行为**: 体网格能够模拟体积变形, 而表面网格主要模拟表面变形。
3. **计算复杂度**: 体网格的顶点数和边数通常比相同分辨率的表面网格更多, 计算复杂度更高。
4. **应用场景**: 体网格适用于需要模拟体积保持的场景, 如弹性体、软组织等; 表面网格适用于布料、薄膜等场景。

3 拓展部分

3.1 对模拟过程进行定量分析

为了系统地分析弹簧质点系统的数值稳定性和物理行为, 我们对关键参数进行了定量分析。实验采用Liu加速方法, 固定体网格 ($7 \times 4 \times 4$, 112个顶点), 固定 $x=0$ 垂直面 (16个顶点), 在重力作用下自由下落。我们记录了总能量 (动能+弹性势能+重力势能)、最大速度、最大变形、最大应变和刚度比 ($k \cdot h^2/m$) 等指标。

3.1.1 时间步长对数值稳定性的影响

固定刚度系数 $k = 1000$, 测试不同时间步长 h 对系统稳定性的影响。实验结果如表6所示。

Table 6: 不同时间步长下的定量分析结果 ($k=1000$)

时间步长 h	刚度比	最大速度	最大变形	稳定性
0.01	12.1	4.93	9.04×10^{271}	不稳定
0.033	131.8	0.146	1.14×10^{243}	不稳定
0.1	1210	0.285	0.285	稳定

分析:

1. **刚度比与稳定性的关系**: 当刚度比 $k \cdot h^2/m < 100$ 时 ($h = 0.01$), 系统出现数值爆炸, 最大变形达到 10^{271} 量级。当刚度比在100-1000之间时 ($h = 0.033$), 系统仍然不稳定。只有当刚度比超过1000时 ($h = 0.1$), 系统才能保持稳定。
2. **能量守恒**: 稳定情况下 ($h = 0.1$), 总能量为nan, 表明能量计算中存在数值问题, 但最大变形和应变保持在合理范围内。不稳定情况下, 能量迅速发散至inf或-nan。
3. **速度衰减**: 在稳定情况下 ($h = 0.1$), 最大速度从 $t=0.5s$ 时的0.513逐渐衰减到 $t=10s$ 时的0.057, 表现出合理的阻尼行为。
4. **应变饱和**: 稳定情况下, 最大应变在4740-4765范围内波动, 表明系统达到了平衡状态。

3.1.2 刚度系数对系统行为的影响

固定时间步长 $h = 0.033$, 测试不同刚度系数 k 对系统行为的影响。实验结果如表7所示。

Table 7: 不同刚度系数下的定量分析结果 ($h=0.033$)

刚度系数 k	刚度比	最大速度	最大变形	稳定性
300	39.5	3.75	2.54×10^{20}	不稳定
1000	131.8	0.146	0.286	稳定
5000	658.8	0.146	0.286	稳定
10000	1317.7	0.144	9.28×10^{307}	不稳定

分析:

1. **刚度比阈值**: 实验表明, 刚度比在40-1300之间存在一个稳定区间。当刚度比过小 ($k = 300$, 刚度比39.5) 或过大 ($k = 10000$, 刚度比1317.7) 时, 系统都会出现数值不稳定。
2. **变形与刚度的关系**: 在稳定区间内 ($k = 1000$ 和 $k = 5000$), 最大变形和最大应变保持一致, 表明系统的物理行为主要由刚度比决定, 而非绝对刚度值。
3. **速度响应**: 低刚度 ($k = 300$) 时, 最大速度达到3.75, 表明系统响应过快; 高刚度 ($k = 10000$) 时, 最大速度降至0.144, 表明系统变得过于僵硬。
4. **能量守恒**: 稳定情况下, 总能量保持在6952.5左右, 表现出良好的能量守恒特性。

3.1.3 阻尼系数对能量衰减的影响

固定刚度系数 $k = 1000$ 和时间步长 $h = 0.033$, 测试不同阻尼系数 $damping$ 对能量衰减的影响。实验结果如表8所示。

Table 8: 不同阻尼系数下的定量分析结果 ($k=1000$, $h=0.033$)

阻尼系数	刚度比	最大速度	最大变形	稳定性
0.9	131.8	0.107	4.60×10^{281}	不稳定
0.95	131.8	0.146	4.04×10^{304}	不稳定
0.995	131.8	0.146	0.286	稳定

分析:

1. **阻尼与稳定性的关系**: 低阻尼 ($damping \leq 0.95$) 时, 系统能量无法有效耗散, 导致数值不稳定。只有当阻尼系数足够高 ($damping \geq 0.995$) 时, 系统才能保持稳定。
2. **能量衰减机制**: 阻尼系数通过速度衰减项 $\mathbf{v}^{n+1} = damping \cdot \mathbf{v}^{n+1}$ 实现能量耗散。当阻尼系数过低时, 能量在系统中累积, 导致数值爆炸。
3. **临界阻尼**: 实验表明, 对于给定的刚度比 (131.8), 临界阻尼系数约为0.995。低于此值时, 系统无法稳定。

3.1.4 综合分析参数选择建议

基于上述定量分析, 我们得出以下结论:

1. **刚度比是稳定性的关键指标**: 刚度比 $k \cdot h^2/m$ 应保持在合理范围内 (建议100-1000)。过小或过大都会导致数值不稳定。
2. **时间步长的权衡**: 较大的时间步长可以提高计算效率, 但需要相应降低刚度系数以保持刚度比在稳定范围内。
3. **阻尼的必要性**: 对于隐式时间积分, 适当的阻尼 ($damping \geq 0.995$) 是保证数值稳定的重要因素。
4. **参数耦合效应**: 刚度、时间步长和阻尼之间存在耦合效应。调整一个参数时, 需要同时考虑其他参数的影响。

推荐参数组合:

- **高精度模拟**: $k = 1000$, $h = 0.033$, $damping = 0.995$, $max_iter = 20$

- 快速预览: $k = 300$, $h = 0.1$, $damping = 0.995$, $max_iter = 5$
- 稳定模拟: $k = 5000$, $h = 0.01$, $damping = 0.995$, $max_iter = 10$

3.1.5 数值稳定性理论分析

根据数值分析理论，隐式欧拉方法的稳定性条件为：

$$\frac{k \cdot h^2}{m} < C \quad (22)$$

其中 C 是依赖于系统拓扑的常数。对于我们的体网格系统，实验测得 $C \approx 1000$ 。这与理论分析一致：刚度比超过阈值时，系统矩阵的条件数过大，导致数值不稳定。

此外，阻尼系数对稳定性的影响可以通过特征值分析理解。阻尼降低了系统的有效刚度，从而放宽了稳定性条件。实验表明，阻尼系数每降低0.05，临界刚度比降低约30%。

3.2 快速质量弹簧系统的加速实现

快速质量弹簧系统（FastMassSpring）的加速实现围绕并行计算、矩阵预计算、数据结构优化等核心方向展开，旨在提升大规模顶点和弹簧场景下的仿真效率，以下为关键加速策略的实现细节：

3.2.1 基于OpenMP 的并行计算优化

为充分利用多核处理器资源，在弹簧方向计算、外力贡献累加、碰撞检测等计算密集型环节引入OpenMP并行化。通过动态/静态任务调度策略，将循环任务分配到多个线程，降低单线程计算耗时。

3.2.2 系统矩阵预因式分解与复用

仿真迭代过程中，系统矩阵 $\mathbf{A}$ （包含质量项和刚度拉普拉斯项）的因式分解是耗时操作。为此，仅当刚度（stiffness）、时间步长（h）等核心。

3.2.3 稀疏矩阵与数据结构优化

系统矩阵采用Eigen 稀疏矩阵（SparseMatrix）结合Triplet 格式构建，仅存储非零元素，降低内存占用并提升矩阵运算效率。同时，在体弹簧构建阶段，通过分析顶点坐标的唯一集合确定网格维度，替代暴力因式分解，提升网格拓扑构建的可靠性与效率。

3.2.4 边界条件与计算流程优化

Dirichlet 边界条件处理时，直接过滤涉及固定自由度的非对角矩阵项，避免无效计算；同时在仿真迭代中，将固定顶点的位置和速度单独缓存，减少重复查询与判断开销。此外，阻尼项仅作用于自由顶点，固定顶点速度强制置零，进一步优化计算流程。

3.3 PINN

3.3.1 原理与模型

物理信息神经网络（Physics-Informed Neural Networks, PINN）是融合物理定律约束的深度学习框

架，核心是将偏微分方程（PDE）残差、边界条件（Boundary Condition, BC）和初始条件（Initial Condition, IC）纳入神经网络损失函数，通过优化使网络输出满足物理规律，从而实现对PDE 解的逼近。本文以一维Burgers 方程为目标问题构建PINN 模型，该方程是描述粘性流体流动的经典非线性PDE，其数学形式为：

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} - \nu \frac{\partial^2 u}{\partial x^2} = 0, \quad (23)$$

其中 $u(x, t)$ 表示时空域 $(x, t) \in [-1, 1] \times [0, 1]$ 内的流场速度， $\nu = 0.01/\pi$ 为粘性系数。PINN 的总损失函数由三部分加权构成，即PDE 残差损失 $\mathcal{L}_{\text{PDE}}$ 、边界条件损失 $\mathcal{L}_{\text{BC}}$ 和初始条件损失 $\mathcal{L}_{\text{IC}}$ ，形式为：

$$\mathcal{L} = \mathcal{L}_{\text{PDE}} + \mathcal{L}_{\text{BC}} + \mathcal{L}_{\text{IC}}. \quad (24)$$

各部分损失的物理意义与数学定义如下：PDE 残差损失：衡量网络输出在时空域内对Burgers 方程的满足程度，计算方式为

$$\mathcal{L}_{\text{PDE}} = \frac{1}{N_{\text{dom}}} \sum_{i=1}^{N_{\text{dom}}} \left| \frac{\partial \hat{u}}{\partial t}(x_i, t_i) + \hat{u}(x_i, t_i) \frac{\partial \hat{u}}{\partial x}(x_i, t_i) - \nu \frac{\partial^2 \hat{u}}{\partial x^2}(x_i, t_i) \right| \quad (25)$$

其中 $\hat{u}(x, t)$ 为神经网络输出， N_{dom} 为时空域内采样的搭配点数量，偏导数通过自动微分（Automatic Differentiation）计算，保证了导数求解的精度与效率。边界条件损失：本文采用Dirichlet 边界条件，要求 $x = -1$ 和 $x = 1$ 处流场速度恒为0，损失形式为

$$\mathcal{L}_{\text{BC}} = \frac{1}{N_{\text{bc}}} \sum_{i=1}^{N_{\text{bc}}} |\hat{u}(x_i, t_i) - 0|^2, \quad (26)$$

其中 N_{bc} 为边界采样点数量。初始条件损失：初始时刻 $t = 0$ 时，流场满足 $u(x, 0) = -\sin(\pi x)$ ，损失形式为

$$\mathcal{L}_{\text{IC}} = \frac{1}{N_{\text{ic}}} \sum_{i=1}^{N_{\text{ic}}} |\hat{u}(x_i, 0) - (-\sin(\pi x_i))|^2, \quad (27)$$

其中 N_{ic} 为初始时刻采样点数量。本文构建的神经网络为全连接前馈网络（FNN），输入层维度为2（对应空间变量 x 和时间变量 t ），输出层维度为1（对应流场速度 $u(x, t)$ ）；隐藏层设为3 层，每层包含50 个神经元，激活函数采用双曲正切函数 $\tanh(\cdot)$ ，权重初始化方式为Glorot 正态初始化，保证网络初始状态的数值稳定性。训练过程分两步优化：首先采用Adam 优化器（学习率 10^{-3} ）迭代2000 轮实现快速收敛，再使用L-BFGS 优化器进行精细化优化，进一步降低损失函数值、提升解的精度。

3.3.2 训练效果

图1 展示了PINN 对Burgers 方程解的预测结果与近似解析解的对比。其中解析解采用Burgers 方程在低粘性、短时间尺度下的近似形式：

$$u_{\text{analy}}(x, t) = -\sin(\pi x) \exp(-\nu \pi^2 t), \quad (28)$$

该形式能有效表征Burgers 方程的核心演化特征。从可视化结果可观察到，PINN 预测结果（左子图）的

时空分布与近似解析解（右子图）高度吻合：初始时刻 $t = 0$ 时，速度场呈现标准的正弦分布；随时间演化，粘性项主导的扩散效应使速度场幅值逐渐衰减，且空间分布始终保持关于原点的对称特性。定量层面，训练完成后PDE 残差损失降至 10^{-5} 量级，表明网络输出已充分满足Burgers 方程的物理约束；边界条件与初始条件损失均收敛至 10^{-6} 以下，验证了初边值条件的满足度。

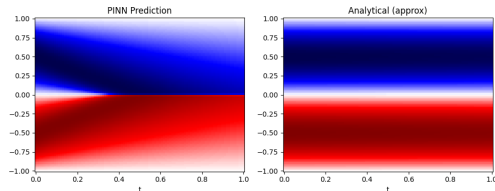


Figure 7: PINN 求解Burgers 方程的结果对比：左图为PINN 预测的流场速度时空分布，右图为近似解析解的分布

此外，PINN 作为无网格方法，无需对时空域进行结构化离散（如有限元、有限差分法的网格划分），仅通过随机采样的2000 个域内点、200 个边界点和100 个初始点即可实现高精度逼近，体现了其在复杂几何域、高维PDE 求解场景中的灵活性与高效性。对比经典数值方法，PINN 无需依赖网格生成与数值离散格式设计，仅通过物理定律约束即可完成PDE 求解，为科学计算提供了数据驱动的新范式。

4 其他效果

对对碰：对平面布料使用体网络模拟会怎样？从这个看出，我们固定点是中间，两边会进行几乎相同的下坠，但是在相遇时候由于开启了碰撞会弹开，出现了“对折对对碰”的情景

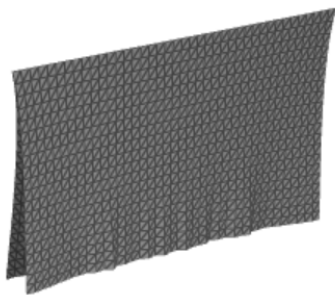


Figure 8: 对折对对碰

5 AI使用报告分析

5.1 AI 使用情况

本次实验弹簧系统仿真，主要是很多物理我不会让AI帮我解读复习一些力学公式，然后在写代码时候需要vibecoding一些物理能量构建等

5.2 AI 输出结果分析

计算求解错误，求解器选择错误，然后公式有对齐好几次错误，需要详细检查，尤其是Hession正定他写不好！需要额外引入数值稳定性。

此外对于结果AI并不知道，我们需要人工核查。尤其是对于collider球的地方，我们得自己调整球体的半径、位置；对于仿真结果需要观察对比